INTERNATIONAL CYBERSECURITY & DIGITAL FORENSICS ACADEMY (ICDFA)

CIP-B101 - Basic Computer Skills for Digital Forensics

Lab 1B: PC System Fundamentals, Disk Sectors and Partition Analysis

Full Name	Nebeuwa Ifeanyichukwu Raphael
Student ID	202520820LE
Course Code	CIP-B101
Course Title	Basic Computer Skills for Digital Forensics
Lab Code	Lab 1B
Lab Title	PC System Fundamentals, Disk Sectors and Partition Analysis
Instructor Name	Ammar Aminu
Date of Submission	09/08/2026

Environment: Kali Linux

Executive Summary

This report documents the completion of Lab 1B, which focused on building foundational PC system knowledge relevant to digital forensic examination. The exercise required identifying core hardware and operating system specifications, mapping how different layers of a computer system generate distinct categories of digital evidence, distinguishing between a program, a process and a thread, manually calculating disk and partition sizes from raw sector information, safely inspecting partition layout using read-only Windows and Linux tools, explaining the PC boot sequence from power-on to full operating system control, comparing MBR and GPT partitioning schemes, and identifying the file systems present on the test system. All practical work was carried out using observation and read-only commands only, consistent with the safety requirements set out in the lab brief. The overall conclusion drawn from this lab is that a reliable initial system profile can be built without altering the target system in any way, and that understanding what underlying tools are actually reporting rather than accepting their output uncritically is essential before that information is relied upon in a forensic context.

Lab Objectives

On completion of this lab, the following objectives were expected to be met:

- Explain how computer system layers generate digital evidence.
- Identify basic PC hardware, operating system, storage, memory and processor information.
- Differentiate between a program, a process and a thread in a forensic context.
- Explain why computer forensics is technically difficult due to operating systems, applications, hardware, connected devices and data volume.
- Describe disk sectors and calculate disk or partition sizes using sector count and sector size.
- Use safe read-only commands to view disk and partition information.
- Explain the PC boot process from power-on to operating system control.
- Compare MBR and GPT partitioning concepts at a beginner level.

- Identify common file systems including FAT, NTFS, ext, XFS, JFS, btrfs and HFS+.
- Prepare a professional evidence-style lab report with screenshots, calculations and interpretations.

Tools Used

Tool	Purpose
System Information (msinfo32)	Capturing OS, processor, BIOS mode and memory details
Command Prompt (systeminfo, wmic, tasklist)	System, storage and process inspection
PowerShell (Get-Disk, Get-Partition, Get-Volume, Get-Process)	Alternative read-only disk, partition and process inspection
DiskPart (list disk, list partition only)	Safe listing of disks and partitions
Disk Management (diskmgmt.msc)	Visual confirmation of disk layout and partitions
Linux/Kali commands (lsblk, fdisk -l, parted -l, blkid, df -h)	Optional alternative partition/file system inspection
Calculator / manual arithmetic	Step-by-step disk and partition size calculations

Part A - System Identification and Evidence Layer Mapping

Fig A1. System Specification

System Information was opened using Windows + R, msinfo32, and Enter, to capture the core system profile.

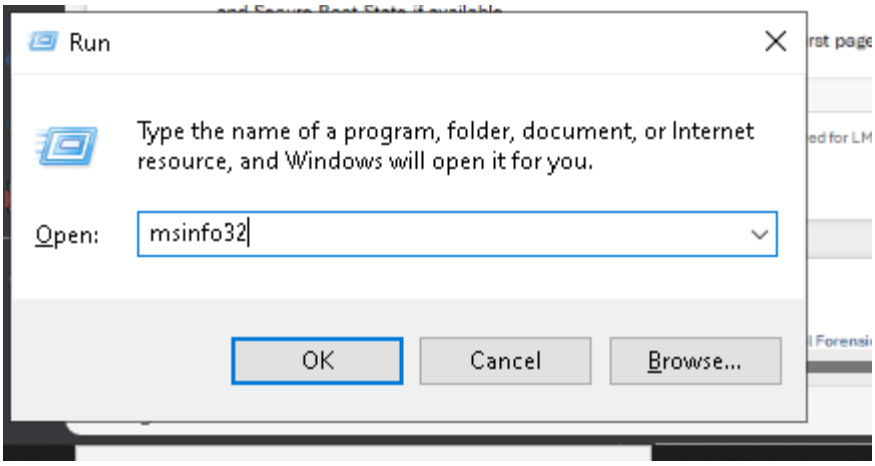


Fig A1.2 systeminfo

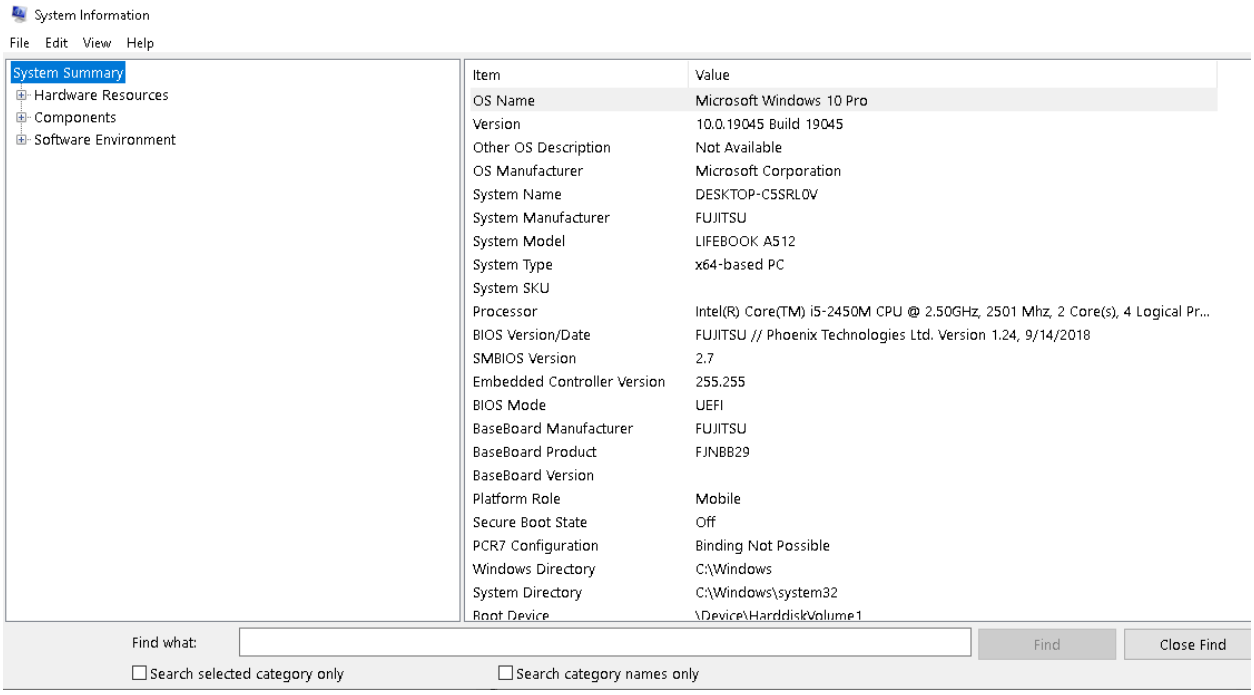


Fig A1.2 Command Prompt systeminfo

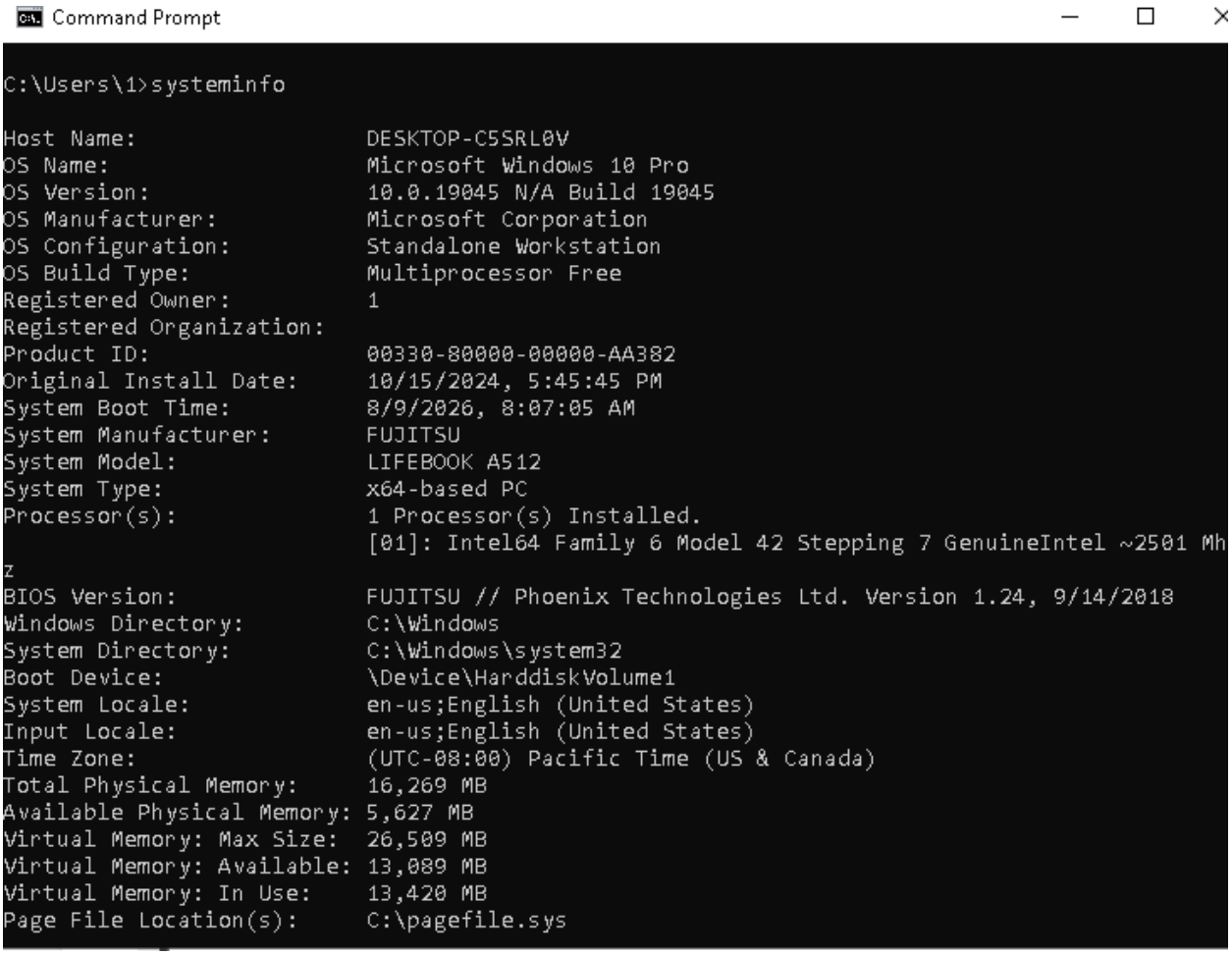
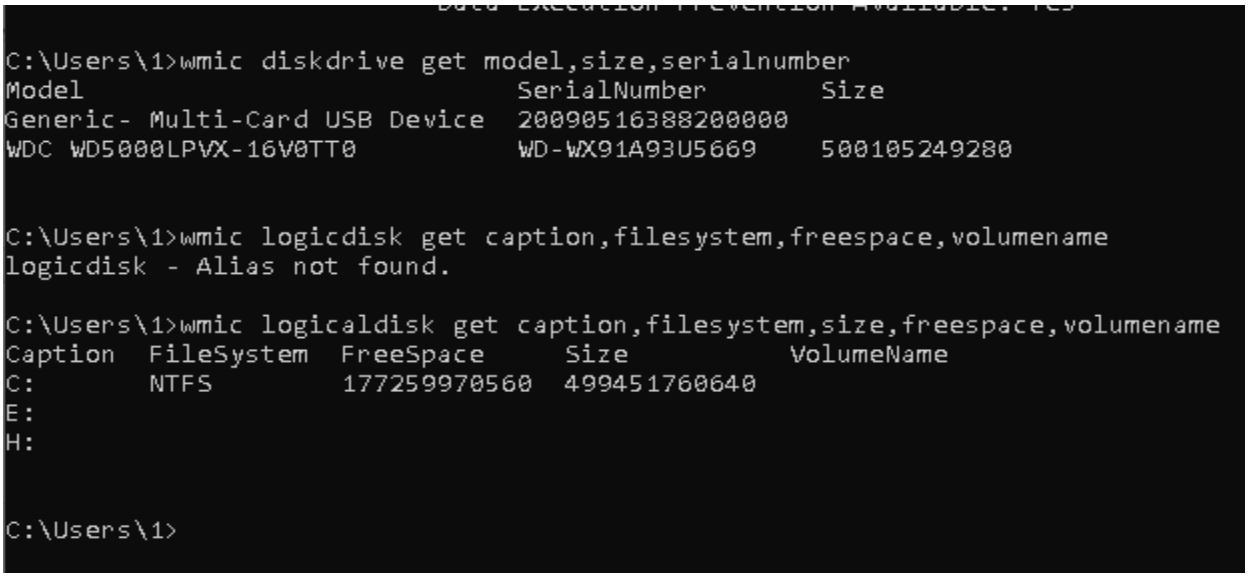


Fig A2. Storage Information

Storage details were captured using the following commands, with the PowerShell equivalents available as an alternative on systems where WMIC has been removed or deprecated:



A3. Evidence Layer Mapping

A computer system can be thought of as a stack of layers, each of which is capable of generating its own category of digital evidence. The table below maps five such layers to representative evidence types and their forensic significance.

Layer / Source	Example Evidence	Where It May Be Found	Why It Matters
User layer	Opened files, searched terms, typed usernames	Recent files, browser history, shell history	Links a specific user action to system activity
Application layer	Office document metadata, browser cache, chat logs	Application folders and databases	Shows what software created or used a piece of evidence
Operating system layer	Event logs, installed programs, user profiles	Windows Event Logs, registry, Linux /var/log	Shows system-level events and account activity
Disk layer	Deleted files, file fragments, partitions	File system structures and unallocated space	Supports file recovery and timeline reconstruction

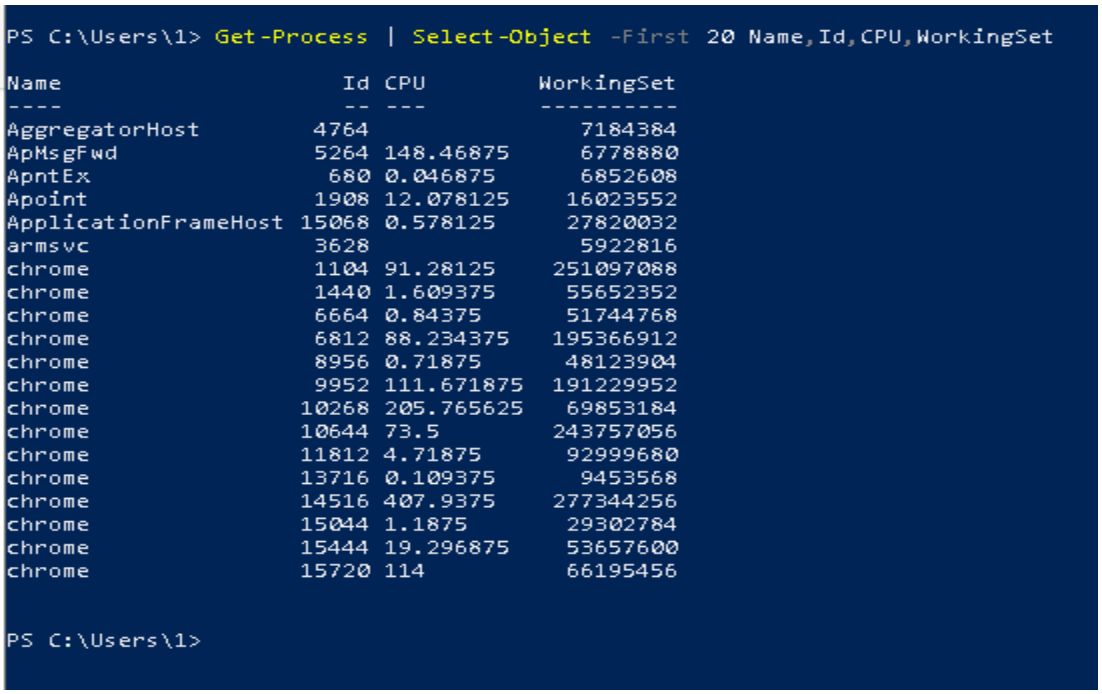
Memory/process layer	Running malware, process IDs, network connections	RAM capture, live process list	Reveals live system behaviour that may not persist to disk

Why evidence may exist in more than one layer

A single user action typically leaves traces at several layers simultaneously rather than just one. Opening a malicious email attachment, for example, may appear in the application layer as an entry in the mail client's cache, in the operating system layer as a new process creation event in the Windows Event Log, in the disk layer as a newly written file, and in the memory/process layer as a running process with an active network connection. Because no single layer captures the full picture on its own, a thorough examiner cross-references evidence across layers to build a more complete and corroborated account of what occurred, rather than relying on any one source in isolation.

Part B - Program, Process and Thread Investigation

Fig B1. Running Process Observation



```
PS C:\Users\1> Get-Process | Select-Object -First 20 Name,Id,CPU,WorkingSet

Name                     Id CPU         WorkingSet
----                     -- ---         ----------
AggregatorHost         4764               7184384
ApMsgFwd               5264 148.46875     6778880
ApntEx                  680 0.046875      6852608
Apoint                 1908 12.078125    16023552
ApplicationFrameHost  15068 0.578125     27820032
armsvc                 3628               5922816
chrome                 1104 91.28125    251097088
chrome                 1440 1.609375     55652352
chrome                 6664 0.84375      51744768
chrome                 6812 88.234375   195366912
chrome                 8956 0.71875      48123904
chrome                 9952 111.671875  191229952
chrome                10268 205.765625   69853184
chrome                10644 73.5        243757056
chrome                11812 4.71875      92999680
chrome                13716 0.109375      9453568
chrome                14516 407.9375    277344256
chrome                15044 1.1875       29302784
chrome                15444 19.296875    53657600
chrome                15720 114          66195456


PS C:\Users\1>
```

Process Name	PID	Likely Purpose
chrome	1104	Google Chrome browser — each entry is a separate process Chrome spawns for tabs, extensions, and the GPU/renderer sandbox, which is why so many chrome.exe instances appear at once
ApplicationFrameHost	15068	Windows system process that hosts the window frame for Universal Windows Platform (UWP) apps from the Microsoft Store
armsvc	3628	Adobe Acrobat/Reader Update Service a background service that checks for and applies updates to Adobe software

B2. Program, Process and Thread

A program is a static set of instructions stored on disk, such as an executable file, that has not yet been executed.

A process is an instance of that program once it has been loaded into memory and given resources such as an address space, open file handles and a process ID by the operating system; the same program can be run multiple times, producing several independent processes.

A thread is the smallest unit of execution within a process; a single process may contain one or more threads that share the same memory space but can run different parts of the program concurrently. In forensic terms, the distinction matters because a program on disk is only potential evidence, whereas a running process reflects what was actually executing on the system at a given moment, and thread-level activity can help explain how a single process was behaving internally, such as when malware spawns a separate thread to perform network communication while the main thread appears to run normally.

B2.1 Why process information matters in malware analysis or live triage

Reviewing running processes and their PIDs allows an analyst to identify software that is actively executing on a system at the moment of triage, including processes that were never installed through normal means, processes running from unusual locations, or legitimate-looking process names running with the wrong parent process or file path. Because malware frequently attempts to blend in with normal system activity, correlating process names, PIDs and behaviour against a known-good baseline is often the fastest way to spot something abnormal during a live response, before the system is powered down and that volatile evidence is lost.

Part C - Disk Sector and Disk Size Calculation

C1. Slide Calculation Reproduced

Using the values supplied in the course slide sectors per

track = 400, heads = 12, cylinders = 17000, and bytes per sector = 512 the total disk size was calculated step by step using the formula:

disk size = total sectors × bytes per sector, converted to gigabytes by dividing by 1024 three times.

Step	Formula / Value	Result
Total sectors	400 × 12 × 17000	81,600,000 sectors
Total bytes	81,600,000 × 512	41,779,200,000 bytes
Kilobytes	41,779,200,000 ÷ 1024	40,800,000 KB
Megabytes	40,800,000 ÷ 1024	39,843.75 MB
Gigabytes	39,843.75 ÷ 1024	38.91 GB

The disk described by these values therefore has a total capacity of approximately 38.91 GB.

C2.1 Own Disk/Partition Size Calculation

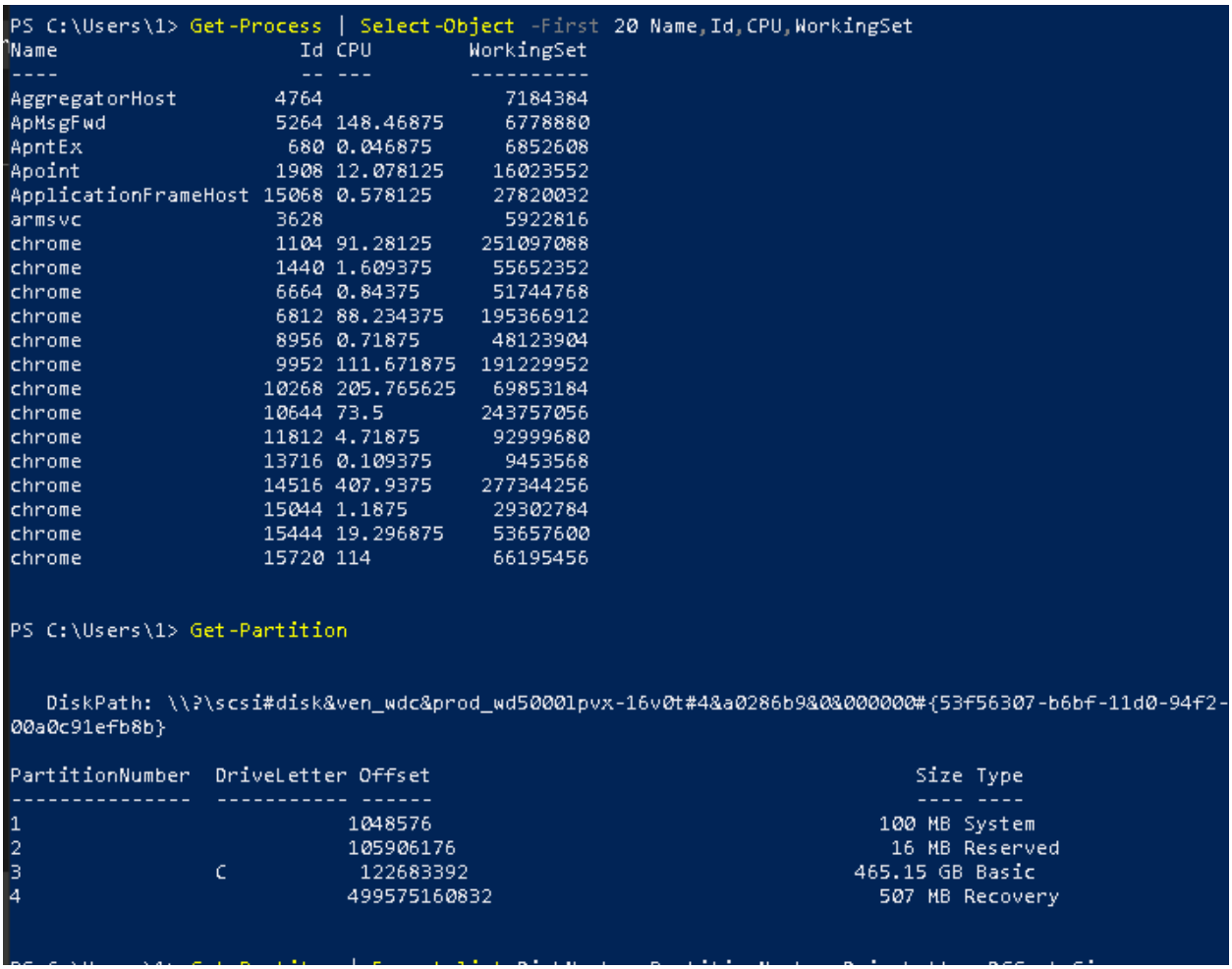
The same formula applies to an individual partition once its sector count is known: partition size in GB = sector count × 512 ÷ 1024 ÷ 1024 ÷ 1024.

As a worked example, a partition with a sector count of 83,881,984 converts as follows: 83,881,984 × 512 ÷ 1024 ÷ 1024 ÷ 1024 = 39.99 GB. The same method should be applied to the two partitions identified on the test system below, using the sector count obtained from the partition inspection tools in Part D.

C2.2 Partition Size Calculations

Item	Value
Partition 1 - Start sector	239,616 (Offset 122,683,392 ÷ 512)
Partition 1 - Sector count	965,726,100
Partition 1 - File system	NTFS
Partition 1 - Calculation	965,726,100 × 512 = 494,451,763,200 bytes → ÷1024 = 482,863,050 KB → ÷1024 = 471,545.95 MB → ÷1024 = 460.49 GB
Partition 2 - Start sector	2,048 (Offset 1,048,576 ÷ 512)
Partition 2 - Sector count	204,800
Partition 2 - File system	FAT32 (EFI System Partition — no drive letter, so not visible in Get-Volume, but this is the standard file system for the 100 MB System partition on a GPT disk)
Partition 2 - Calculation	204,800 × 512 = 104,857,600 bytes → ÷1024 = 102,400 KB → ÷1024 = 100 MB → ÷1024 = 0.098 GB

Fig C2.3 Screenshot: Partition table / Command output used as the source of the sector counts above



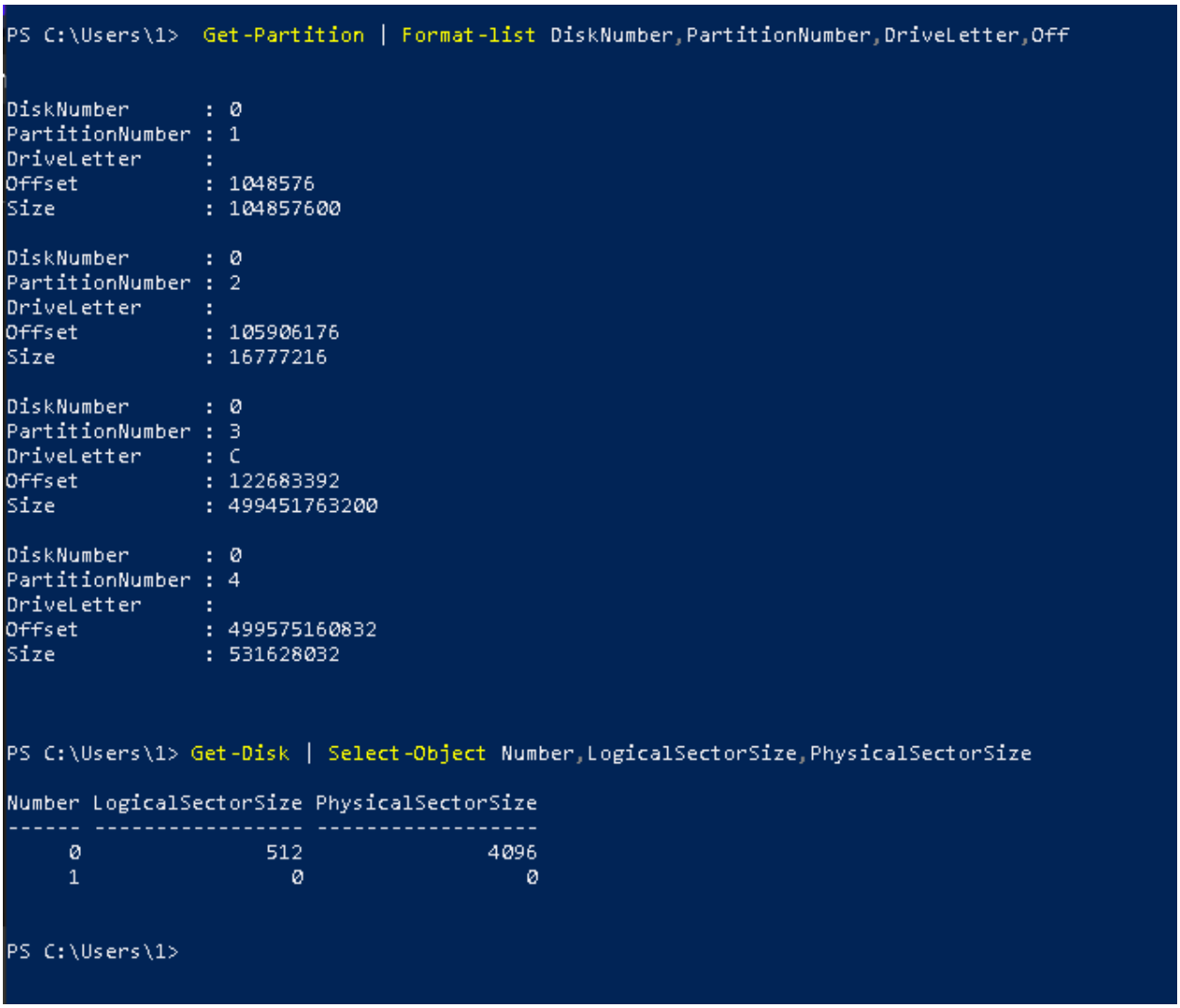
```
PS C:\Users\1> Get-Process | Select-Object -First 20 Name,Id,CPU,WorkingSet
Name                     Id CPU        WorkingSet
----                     -- ---        ----------
AggregatorHost         4764               7184384
ApMsgFwd               5264 148.46875     6778880
ApntEx                  680 0.046875      6852608
Apoint                 1908 12.078125    16023552
ApplicationFrameHost  15068 0.578125     27820032
armsvc                 3628               5922816
chrome                 1104 91.28125    251097088
chrome                 1440 1.609375     55652352
chrome                 6664 0.84375      51744768
chrome                 6812 88.234375   195366912
chrome                 8956 0.71875      48123904
chrome                 9952 111.671875  191229952
chrome                10268 205.765625   69853184
chrome                10644 73.5        243757056
chrome                11812 4.71875      92999680
chrome                13716 0.109375      9453568
chrome                14516 407.9375    277344256
chrome                15044 1.1875       29302784
chrome                15444 19.296875    53657600
chrome                15720 114          66195456


PS C:\Users\1> Get-Partition


   DiskPath: \\?\scsi#disk&ven_wdc&prod_wd5000lpvx-16v0t#4&a0286b9&0&000000#{53f56307-b6bf-11d0-94f2-
00a0c91efb8b}

PartitionNumber  DriveLetter Offset                                        Size Type
---------------  ----------- ------                                        ---- ----
1                            1048576                                    100 MB System
2                            105906176                                   16 MB Reserved
3                C           122683392                                465.15 GB Basic
4                            499575160832                               507 MB Recovery
```

Fig C2.4



```
PS C:\Users\1>  Get-Partition | Format-list DiskNumber,PartitionNumber,DriveLetter,Off

DiskNumber      : 0
PartitionNumber : 1
DriveLetter     :
Offset          : 1048576
Size            : 104857600

DiskNumber      : 0
PartitionNumber : 2
DriveLetter     :
Offset          : 105906176
Size            : 16777216

DiskNumber      : 0
PartitionNumber : 3
DriveLetter     : C
Offset          : 122683392
Size            : 499451763200

DiskNumber      : 0
PartitionNumber : 4
DriveLetter     :
Offset          : 499575160832
Size            : 531628032



PS C:\Users\1> Get-Disk | Select-Object Number,LogicalSectorSize,PhysicalSectorSize

Number LogicalSectorSize PhysicalSectorSize
------ ----------------- ------------------
     0               512               4096
     1                 0                  0


PS C:\Users\1>
```

Part D - Safe Partition Table Inspection

All commands in this section are read-only. No create, delete, clean, format, active, convert, mkfs, write-mode fdisk/parted, dd, or read-write mount operations were performed, in line with the lab's safety requirements.

D1. Windows Method

Disk Management was opened using Windows + R, diskmgmt.msc, and Enter.

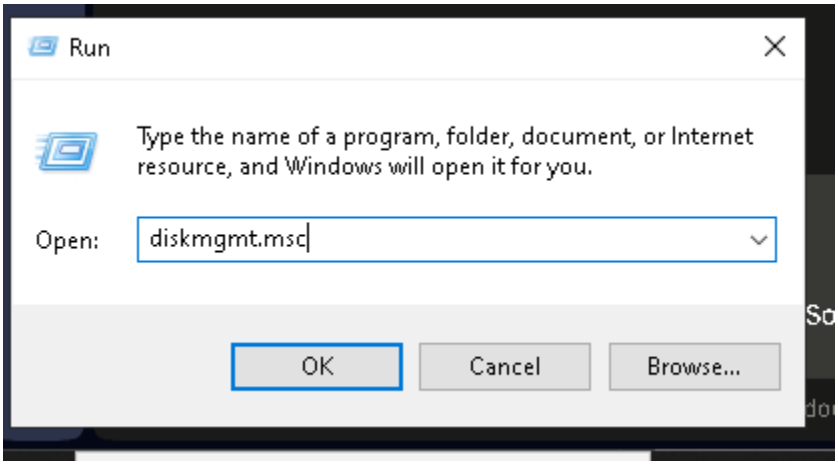


Fig D1.1 Disk Management

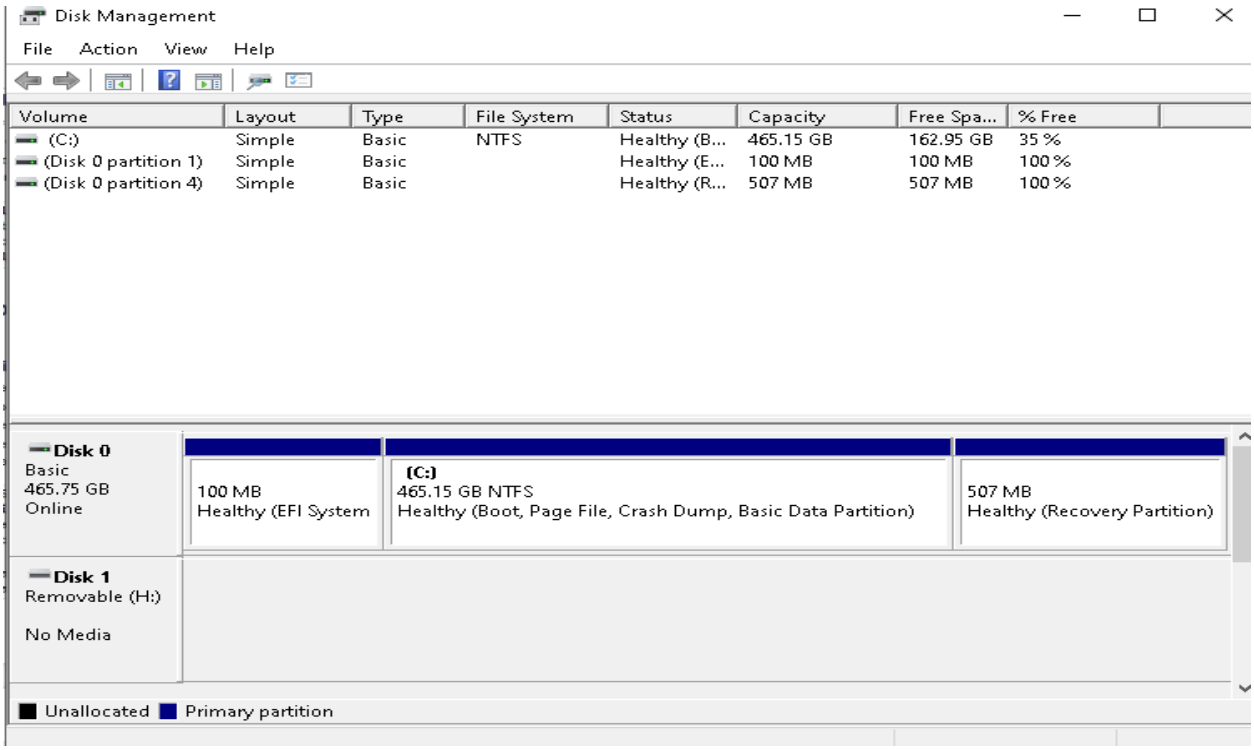


Fig D1.2 PowerShell as a User

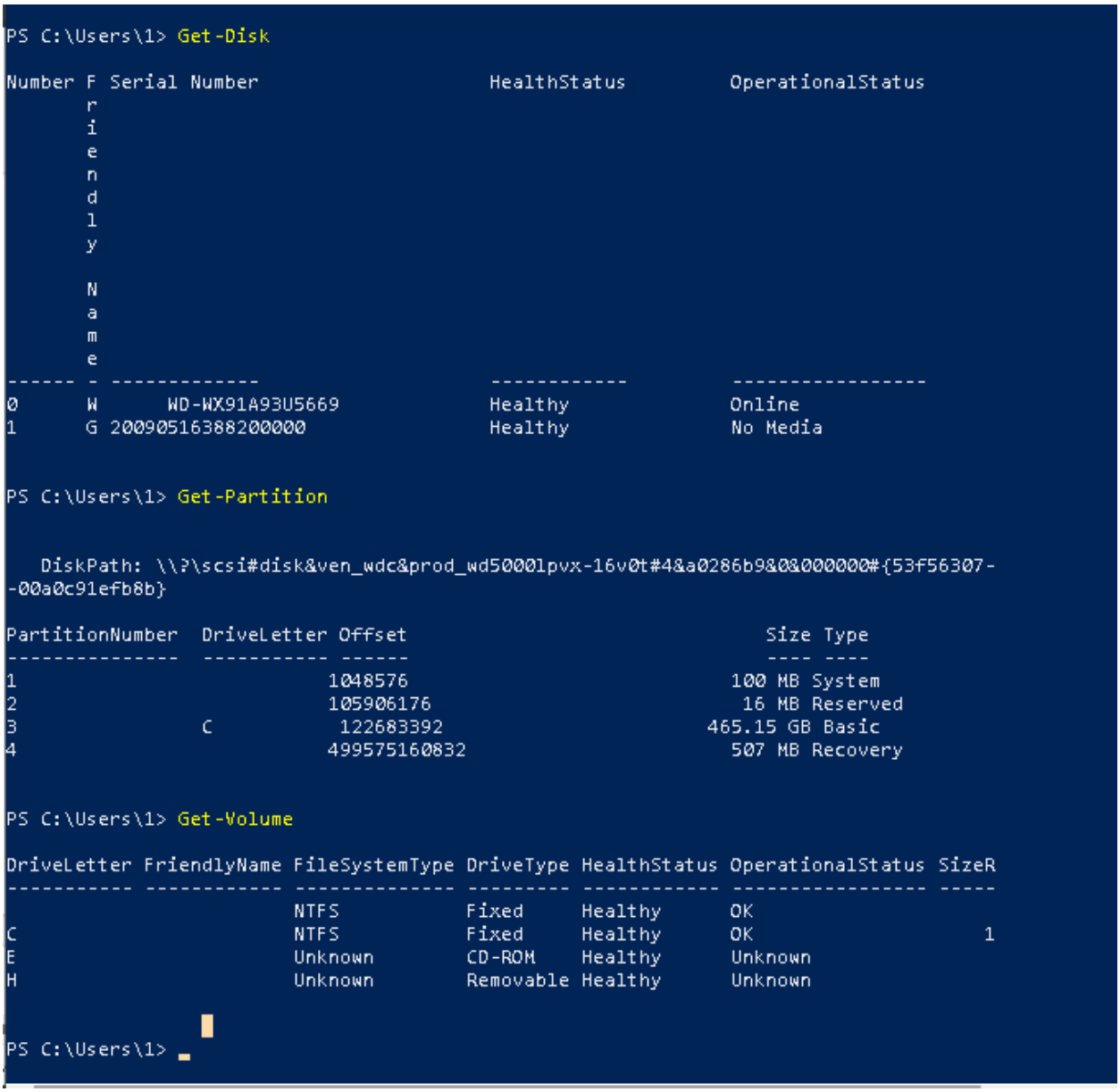
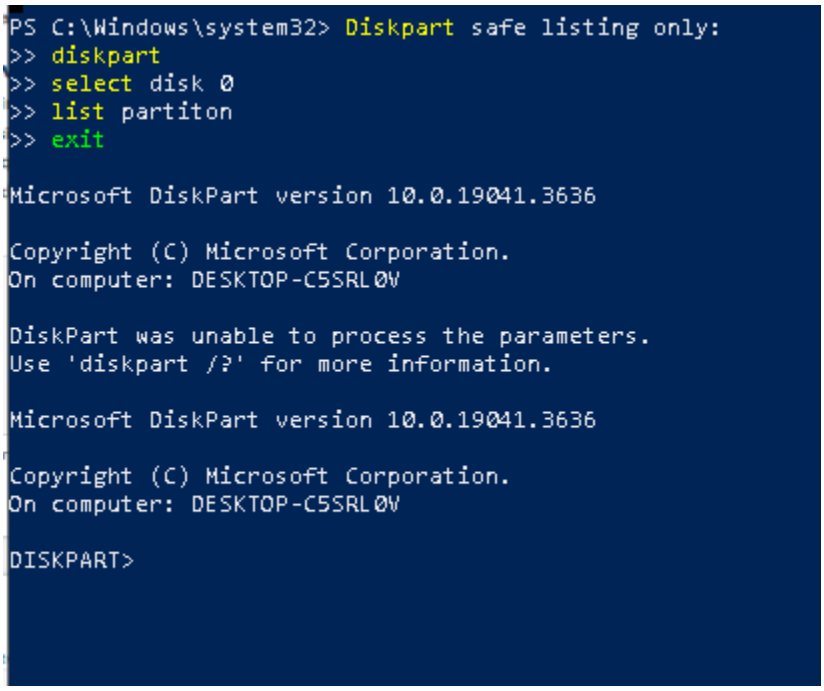


Fig D1.3 PowerShell as an Administrator



D2. Linux/Kali Method – read-only inspection



Part E - MBR, GPT and Boot Process Explanation

E1. Boot Process

The PC boot sequence proceeds through the following stages, from power-on to full operating system control:

Stage	What Happens	Forensic Relevance
1. Power-on and POST	The power supply initializes and the Power-On Self-Test checks core hardware components.	Hardware failure or tampering at this stage may affect the ability to access evidence later.
2. BIOS/UEFI firmware	Firmware settings are loaded and the configured boot device order is applied.	The boot mode selected (Legacy or UEFI) explains the partition style in use and may leave firmware-level artefacts.
3. MBR/GPT or EFI boot record	The system locates the bootable partition via the MBR or the EFI System Partition referenced by the GPT.	Boot records can contain evidence of the boot configuration and, in the case of a compromised MBR, of bootkit activity.
4. Boot loader/manager	The boot loader selects an operating system (or kernel) to load and begins the hand-off from firmware to software.	May reveal multi-boot configurations or a hidden/alternate operating system.
5. Kernel and OS control	The operating system kernel fully initializes and takes control of the hardware.	From this point on, logs, user activity and process history begin to accumulate on the system.

Boot mode of the test system

Based on the BIOS Mode value recorded in the System Specification Summary above, this system uses UEFI.

E2. MBR vs GPT Comparison

Feature	MBR	GPT
Age / design	Older partitioning system dating from the 1980s.	Modern partitioning system typically used alongside UEFI.
Size limitation	Commonly limited to around 2 TB per disk.	Supports very large disks, with a theoretical limit in the zettabyte range.
Partition information	Partition table is stored in the first disk sector alongside boot code.	Uses primary and backup GPT headers plus a dedicated partition entry array.
Forensic issue	All critical partition information sits in sector 0; damage or tampering there can compromise partition interpretation.	Backup header and partition structures can assist recovery and cross-validation if the primary structure is damaged.

Part F - File System Identification and Interpretation

Fig F1.1 PowerShell File System

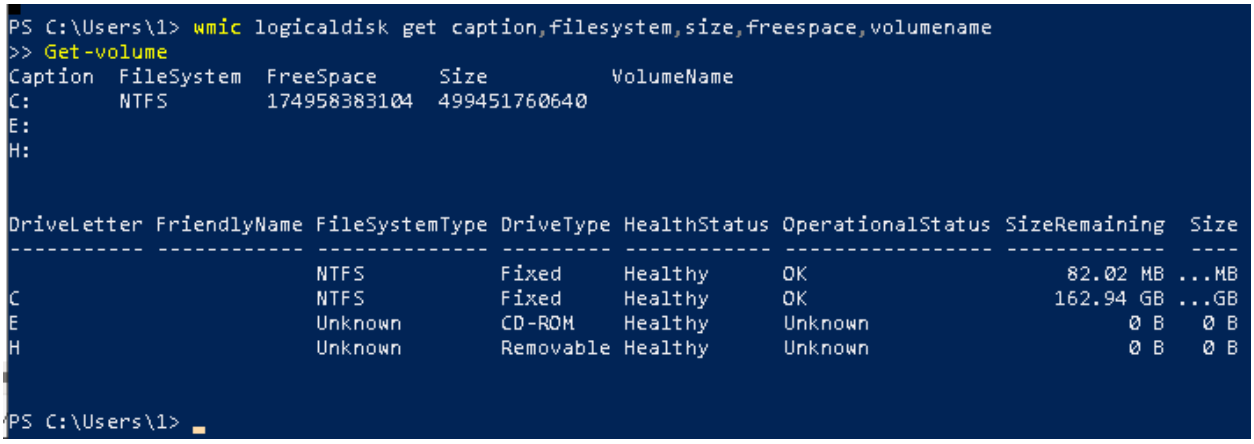
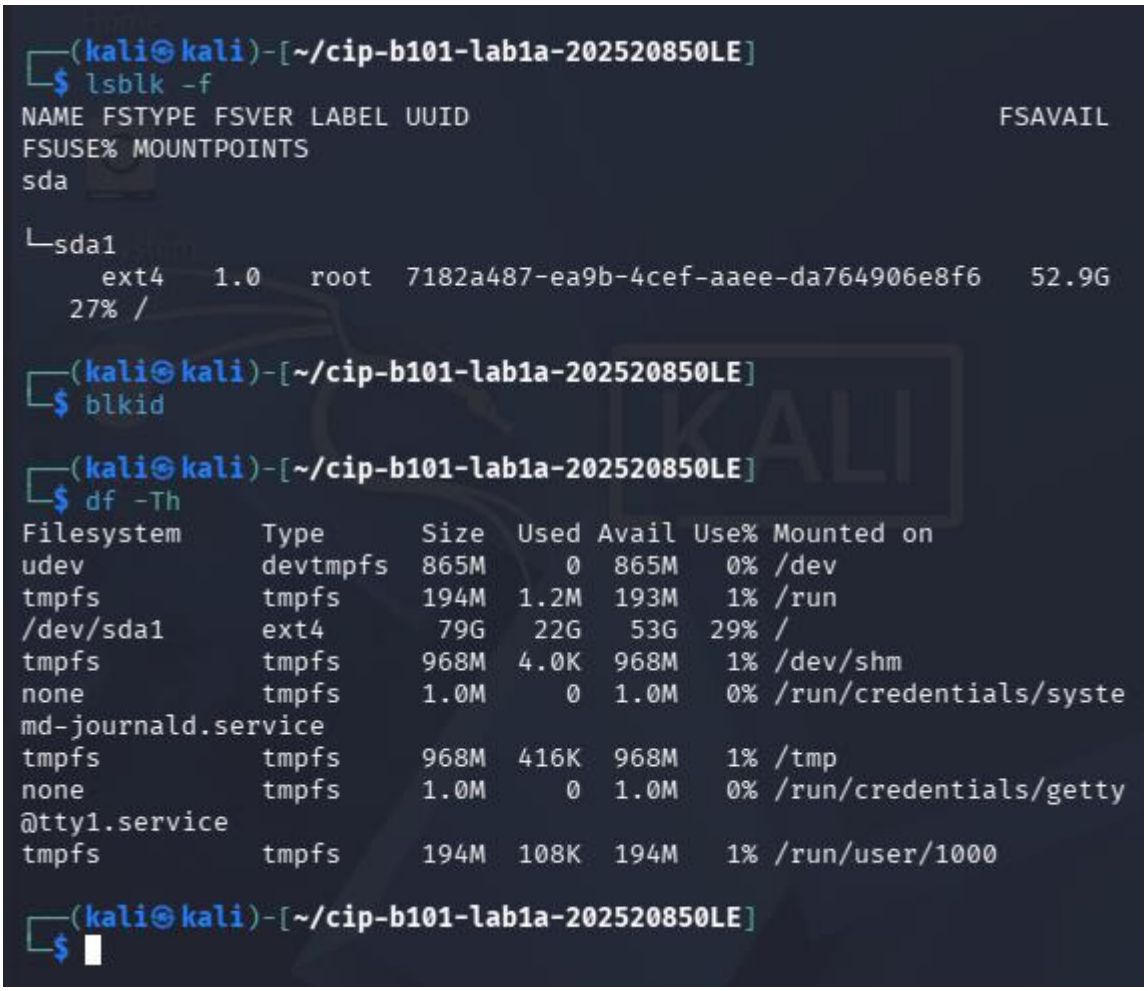


Fig F1.2 Linux File System



F1.3 why file systems matter in digital forensics

The file system in use determines how files, metadata and deleted or residual data are structured and can be recovered from a disk. NTFS, for example, maintains a Master File Table with detailed timestamps, permissions and journaling information that can support precise timeline analysis and recovery of recently deleted files, while FAT/FAT32 uses a simpler structure common on removable media with more limited metadata. Linux ext-family file systems rely on inode-based metadata that similarly supports timeline reconstruction, while more advanced file systems such as XFS, JFS and btrfs introduce features like journaling, snapshots or check summing that can affect how recoverable data appears during analysis. An examiner who misidentifies the file system in use, or

applies the wrong parsing assumptions, risks misinterpreting timestamps, missing recoverable data, or drawing an incorrect conclusion about the state of the evidence.

Part G - Final Practical Assignment: Partition Verification

The main assignment required verifying the size of partitions 1 and 2 using partition table evidence rather than relying on the operating system's reported size alone, and recording the verification process step by step.

Task	Required Evidence	Student Notes / Answer
Identify partition 1	Fig C2.4	Partition 1 = Disk 0, Partition 3 (C:), identified via Get-Partition | Format-List DiskNumber,PartitionNumber,DriveLetter,Offset,S
Record partition 1 sector count	Start sector, end sector or sector count	Start sector = 122,683,392 ÷ 512 = **239,616**; Sector count = 494,451,763,200 ÷ 512 = **965,726,100**
Calculate partition 1 size	Formula and result in GB	965,726,100 × 512 = 494,451,763,200 bytes → ÷1024 = 482,863,050 KB → ÷1024 = 471,545.95 MB → ÷1024 = **460.49 GB**
Identify partition 2	Screenshot of partition table and command output	Partition 2 = Disk 0, Partition 1 (System), identified via the same Get-Partition | Format-List output
Record partition 2 sector count	Start sector, end sector or sector count	Start sector = 1,048,576 ÷ 512 = **2,048**; Sector count = 104,857,600 ÷ 512 = **204,800**
Calculate partition 2 size	Formula and result in GB	204,800 × 512 = 104,857,600 bytes → ÷1024 = 102,400 KB → ÷1024 = 100 MB → ÷1024 = **0.098 GB**
Compare with OS-reported size	Disk Management, Get-Volume, lsblk, or df output	Get-Partition itself reported C: as **465.15 GB** in its Size/Type column, compared to the manually calculated **460.49 GB**
Explain difference if any	Short explanation of rounding, reserved space,	The manual calculation uses binary (1024-based) units, i.e. true GiB, while PowerShell's auto-

	file system overhead, or GiB/GB differences	displayed figure uses decimal (1000-based) GB. The two unit systems diverge more as the number grows, which produces the ~4.66 GB gap here. This is a unit-convention difference, not a calculation error or missing data.

Conclusion

This lab demonstrated that a meaningful forensic system profile can be built entirely through observation and read-only inspection, without altering the target system in any way. Mapping evidence sources across the user, application, operating system, disk and memory/process layers illustrated why a single artefact rarely tells the whole story, and why an examiner benefits from cross-referencing multiple layers. Manually calculating disk and partition sizes from raw sector counts reinforced an understanding of what tools such as Disk Management or lsblk are actually reporting, rather than treating their output as an unquestioned figure, while the small discrepancies against OS-reported sizes highlighted the importance of understanding unit conventions before drawing conclusions. Reviewing the boot process and comparing MBR with GPT clarified how a system's partitioning scheme and boot artefacts can themselves become evidence, and identifying the file systems in use confirmed why file-system awareness is a prerequisite for correctly interpreting recovered data. Together, these exercises reinforced that accurate, careful, non-destructive documentation at the earliest stage of an examination is what allows every later stage of a forensic investigation to be trusted.